



Lab AWS Lambda

Cho người mới · từ Hello World đến production · 8 phần thực hành · Python 3.12

PHẦN 01 Hàm Lambda đầu tiên

Dễ · 15 phút

💡 Hãy hình dung Lambda như một nhân viên chỉ làm việc đúng lúc được gọi — không gọi thì không tốn tiền. Ta sẽ “thuê” bạn ấy nói một lời chào.

1 Mở trình tạo function

Đăng nhập AWS Console, tìm dịch vụ Lambda rồi bấm Create function.

Console > Lambda > Functions

Trên AWS Console, bấm →

Create function

2 Cấu hình cơ bản

Chọn “Author from scratch”. Đặt tên là hello-world, Runtime chọn Python 3.12, các mục khác giữ mặc định.

Create function > Author from scratch

Trên AWS Console, bấm →

Create function

3 Viết code cho handler

Ở tab Code, xoá code mẫu và dán đoạn dưới. `lambda_handler` chính là “cửa vào” — AWS gọi hàm này mỗi khi function chạy.

`lambda_function.py`

```
def lambda_handler(event, context):
    return {
        'statusCode': 200,
        'body': 'Xin chào từ Lambda!'
    }
```

4

Deploy và chạy thử

Bấm Deploy để lưu, rồi bấm Test, tạo một test event rỗng và bấm Test lần nữa. Bạn sẽ thấy kết quả trả về!

Code > Deploy

Trên AWS Console, bấm →

Test

✦ Lần chạy đầu có thể chậm ~1 giây (gọi là “cold start”). Các lần sau sẽ nhanh hơn nhiều.

🧠 `lambda_handler` dùng để làm gì?

A Là hàm AWS tự động gọi khi function chạy

B Là tên file bắt buộc của Python

C Chỉ để in log ra màn hình

D Là một loại cơ sở dữ liệu

✅ Đáp án A. Handler là điểm bắt đầu — AWS truyền event và context vào đây mỗi lần function được kích hoạt.

PHẦN 02 Tạo REST API với API Gateway

Trung bình • 25 phút

💡 Function mới chỉ chạy được trong AWS. API Gateway sẽ gán cho nó một địa chỉ URL để cả thế giới gọi được qua trình duyệt.

1 Thêm trigger API Gateway

Mở function hello-world. Bấm Add trigger và chọn API Gateway.

Function > Add trigger

Trên AWS Console, bấm →

API Gateway

2 Chọn kiểu API

Chọn Create a new API > HTTP API. Ở Security chọn Open (để học; thực tế nên bảo vệ). Bấm Add.

Add trigger > HTTP API > Security: Open

Trên AWS Console, bấm →

Add

3 Đọc tham số từ URL

Cập nhật code để đọc ?name=... từ URL và trả về JSON. Nhớ bấm Deploy lại sau khi sửa.

lambda_function.py

```
import json

def lambda_handler(event, context):
    p = event.get('queryStringParameters') or {}
    name = p.get('name', 'bạn')
    return {
        'statusCode': 200,
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'message': f'Chào {name}!'})
    }
```

4 Mở thử trên trình duyệt

Ở Configuration > Triggers, copy Invoke URL, thêm ?name=An vào cuối rồi mở trên trình duyệt.

Configuration > Triggers > API endpoint

Trên AWS Console, bấm →

Mở URL ↗

✦ Thử đổi tên trong ?name= để thấy phản hồi thay đổi ngay lập tức.

 API Gateway giúp gì cho Lambda?

- A** Tạo một địa chỉ URL để gọi Lambda qua HTTP
- B Tăng RAM cho function
- C Lưu trữ file
- D Gửi email tự động

☒ Đáp án A. API Gateway biến function thành một endpoint HTTP mà trình duyệt hay ứng dụng đều gọi được.

💡 Lambda có thể tự thức dậy khi có sự kiện. Ở đây, mỗi khi ai đó upload file lên S3, function sẽ tự chạy để xử lý.

1 Tạo bucket S3

Vào S3 > Create bucket, đặt tên duy nhất (vd my-lab-uploads-anhtu). Các mục khác để mặc định.

Console > S3 > Buckets

Trên AWS Console, bấm →

Create bucket

2 Gắn trigger S3 vào Lambda

Về Lambda, Add trigger > S3, chọn bucket vừa tạo, Event type: All object create events.

Function > Add trigger > S3

Trên AWS Console, bấm →

Add

3 Đọc thông tin file từ event

Khi có file upload, S3 gửi một event chứa tên bucket và key. Dán code để in ra.

lambda_function.py

```
import urllib.parse

def lambda_handler(event, context):
    rec = event['Records'][0]['s3']
    bucket = rec['bucket']['name']
    key = urllib.parse.unquote_plus(rec['object']['key'])
    print(f'File mới: {key} (bucket: {bucket})')
    return {'statusCode': 200}
```

✦ unquote_plus giúp giải mã tên file có dấu cách hoặc ký tự đặc biệt.

4 Kiểm tra bằng cách upload

Upload một file bất kỳ lên bucket, sau đó xem CloudWatch Logs để thấy tên file được in ra.

S3 > bucket > Upload

Trên AWS Console, bấm →

Upload



Khi S3 kích hoạt Lambda, thông tin file nằm ở đâu trong event?

A `event['Records'][0]['s3']`

B `event.file`

C `event.body`

D Không có, phải tự truy vấn



Đáp án A. S3 đóng gói chi tiết đối tượng trong mảng Records → s3 → bucket / object.

💡 Cần lưu dữ liệu mà không muốn quản lý máy chủ database? DynamoDB là kho lưu trữ serverless ghép cực hợp với Lambda.

1 Tạo bảng DynamoDB

Vào DynamoDB › Create table. Tên bảng Visitors, Partition key đặt là id (kiểu String).

Console › DynamoDB › Tables

Trên AWS Console, bấm →

Create table

2 Cấp quyền cho Lambda

Vào Configuration › Permissions, mở Role và gắn policy AmazonDynamoDBFullAccess (để học; thực tế nên giới hạn quyền).

Configuration › Permissions › Role

Trên AWS Console, bấm →

Attach policies

✦ boto3 là thư viện AWS có sẵn trong runtime Python — không cần cài thêm.

3 Ghi và đọc dữ liệu

Dán code dùng boto3 để ghi một mục rồi đọc lại. Bấm Deploy & Test.

lambda_function.py

```
import boto3

table = boto3.resource('dynamodb').Table('Visitors')

def lambda_handler(event, context):
    table.put_item(Item={'id': 'trang-chu', 'luot': 1})
    resp = table.get_item(Key={'id': 'trang-chu'})
    return {'statusCode': 200, 'body': str(resp['Item'])}
```

4 Xem dữ liệu trong bảng

Mở lại DynamoDB › bảng Visitors › Explore items để thấy dòng vừa được ghi.

DynamoDB › Visitors › Explore items

Trên AWS Console, bấm →

Run scan



Partition key trong DynamoDB là gì?

- A** Khoá chính để định danh và phân bổ dữ liệu
- B Mật khẩu của bảng
- C Số dòng tối đa
- D Tên vùng AWS

✅ Đáp án A. Partition key xác định duy nhất mỗi mục và quyết định dữ liệu được lưu ở phân vùng nào.

PHẦN 05 Gửi thông báo với SNS

Trung bình · 20 phút

💡 SNS giúp function gửi email hoặc tin nhắn cho người dùng — ví dụ gửi cảnh báo khi có sự cố xảy ra.

1 Tạo SNS Topic

Vào SNS › Topics › Create topic, chọn kiểu Standard, đặt tên canh-bao.

Console › SNS › Topics

Trên AWS Console, bấm →

Create topic

2 Đăng ký nhận email

Trong topic, bấm Create subscription, Protocol: Email, nhập email của bạn. Vào hộp thư bấm xác nhận.

Topic canh-bao › Create subscription

Trên AWS Console, bấm →

Create subscription

✦ Phải bấm link xác nhận trong email thì mới bắt đầu nhận được thông báo.

3 Publish thông báo từ Lambda

Cấp quyền AmazonSNSFullAccess cho role, thay ARN topic vào code rồi Deploy & Test.

lambda_function.py

```
import boto3

sns = boto3.client('sns')
TOPIC_ARN = 'arn:aws:sns:REGION:ACCOUNT:canh-bao'

def lambda_handler(event, context):
    sns.publish(
        TopicArn=TOPIC_ARN,
        Subject='Cảnh báo',
        Message='Có sự kiện mới!'
    )
    return {'statusCode': 200}
```

4 Kiểm tra hộp thư

Sau khi bấm Test, mở email của bạn — bạn sẽ nhận được thông báo “Có sự kiện mới!”.

Configuration › Permissions

Trên AWS Console, bấm →

Attach SNS policy

 SNS chủ yếu dùng để làm gì?

- A** Gửi thông báo (email / SMS) đến người đăng ký
- B** Lưu file lớn
- C** Chạy container
- D** Tạo cơ sở dữ liệu

✓ Đáp án A. SNS là dịch vụ pub/sub: bạn publish một message, mọi subscriber (email, SMS, Lambda...) đều nhận được.

PHẦN 06 Lên lịch chạy tự động (EventBridge)

Trung bình - 20 phút

💡 Không phải lúc nào cũng cần người bấm nút. EventBridge cho phép hẹn giờ để Lambda tự chạy định kỳ như một chiếc đồng hồ báo thức.

1 Mở EventBridge Scheduler

Vào Amazon EventBridge › Schedules › Create schedule.

Console › EventBridge › Schedules

Trên AWS Console, bấm →

Create schedule

2 Đặt lịch chạy

Chọn Recurring schedule. Dùng rate(5 minutes) để chạy mỗi 5 phút, hoặc cron(0 9 * * ? *) để chạy 9h sáng mỗi ngày.

Create schedule › Recurring

Trên AWS Console, bấm →

Next

✦ cron của AWS có 6 trường: phút, giờ, ngày, tháng, thứ, năm.

3 Chọn Lambda làm mục tiêu

Ở Target chọn AWS Lambda › Invoke và chọn function của bạn. Dán code in ra thời điểm chạy.

lambda_function.py

```
from datetime import datetime

def lambda_handler(event, context):
    print(f'Chạy định kỳ lúc {datetime.now()}')
    return {'statusCode': 200}
```

4 Chờ và kiểm tra log

Sau khi tạo xong, đợi tới lịch rồi mở CloudWatch Logs để thấy dòng “Chạy định kỳ...”.

EventBridge › Schedule › Monitoring

Trên AWS Console, bấm →

Xem log

 **cron(0 9 * * ? *) nghĩa là chạy khi nào?**

A 9:00 sáng mỗi ngày

B Mỗi 9 phút

C 9 giờ tối thứ Hai

D Mỗi 9 giờ một lần

☒ Đáp án A. 0 phút, 9 giờ, mọi ngày / tháng / thứ → đúng 9:00 sáng hằng ngày.

PHẦN 07 Biến môi trường & bảo mật IAM

Khó - 25 phút

💡 Đây là phần “người lớn”: giữ bí mật an toàn bằng biến môi trường và chỉ cấp đúng quyền cần thiết theo nguyên tắc least privilege.

1 Thêm biến môi trường

Vào Configuration > Environment variables > Edit. Thêm API_KEY và STAGE. Giá trị nằm ngoài code nên an toàn hơn.

Configuration > Environment variables

Trên AWS Console, bấm →

Edit

2 Đọc biến trong code

Dùng os.environ để đọc. Dán code, Deploy & Test.

lambda_function.py

```
import os

def lambda_handler(event, context):
    api_key = os.environ['API_KEY']
    stage = os.environ.get('STAGE', 'dev')
    return {'statusCode': 200, 'body': f'Đang chạy ở stage={stage}'}
```

✦ Dùng .get(key, default) để tránh lỗi khi biến chưa được đặt.

3 Nguyên tắc quyền tối thiểu (IAM)

Mở Role của function. Thay vì FullAccess, hãy chỉ cấp đúng action cần dùng như policy ví dụ dưới đây.

IAM policy – quyền tối thiểu (JSON)

```
{
  "Effect": "Allow",
  "Action": ["dynamodb:PutItem", "dynamodb:GetItem"],
  "Resource": "arn:aws:dynamodb:*:*:table/Visitors"
}
```

4 Kiểm tra lại

Bấm Test. Nếu thiếu quyền, log sẽ báo AccessDenied — đó chính là cách IAM bảo vệ tài nguyên của bạn.

Code > Test

Trên AWS Console, bấm →

Test

 **Vì sao nên dùng biến môi trường thay vì ghi thẳng khoá vào code?**

A Tránh lộ bí mật và dễ đổi cấu hình theo môi trường

B Giúp code chạy nhanh hơn

C Bắt buộc bởi Python

D Để tốn ít RAM hơn

☒ Đáp án A. Khoá bí mật không nên nằm trong mã nguồn; biến môi trường giúp tách cấu hình và giữ an toàn.

PHẦN 08 Xem log & gỡ lỗi với CloudWatch

Trung bình - 20 phút

💡 Khi có lỗi, log là người bạn tốt nhất. Mọi thứ function in ra đều được lưu ở CloudWatch để bạn soi lại bất cứ lúc nào.

1 Mở CloudWatch Logs

Trong function vào Monitor › View CloudWatch logs, hoặc CloudWatch › Log groups › /aws/lambda/ten-function.

Function › Monitor

Trên AWS Console, bấm →

[View CloudWatch logs](#)

2 Ghi log có cấp độ

Dùng module logging thay cho print để có INFO / ERROR rõ ràng. Dán code, Deploy & Test.

lambda_function.py

```
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    logger.info('Bắt đầu xử lý')
    try:
        ket_qua = 1 / 0
    except Exception as e:
        logger.error(f'Đã xảy ra lỗi: {e}')
    return {'statusCode': 200}
```

✦ print() cũng vào CloudWatch, nhưng logging cho bạn phân loại mức độ và lọc dễ hơn.

3 Đọc log stream

Mở log group, chọn log stream mới nhất để xem từng dòng của lần chạy vừa rồi, kèm lỗi nếu có.

Log group › Log streams

Trên AWS Console, bấm →

[Xem stream mới nhất](#)

4

Xem Metrics

Tab Monitor có biểu đồ Invocations, Errors, Duration — giúp bạn theo dõi “sức khỏe” của function.

Function > Monitor > Metrics

Trên AWS Console, bấm →

Xem biểu đồ



Khi bạn dùng print() trong Lambda, output đi đâu?

- A** Vào CloudWatch Logs
- B Bị bỏ qua hoàn toàn
- C Hiện trên màn hình máy bạn
- D Được gửi vào email

✓ Đáp án A. Mọi stdout / stderr của function được tự động chuyển vào CloudWatch Logs.